

Presentación:

Nombre:

Algelis Severino

Tema:

Introducción a la Programación y Diagrama de flujo.

Tecnico:

Ciberseguridad Semipresencial

Facilitador:

Kevin Feliz Henriquez

Centro:

Diapiato Pro salud

Descripción:

En este documento .pdf adjunto capturas del reto 8 de 30 days of code de HackerRank.

Link GitHub:

<https://github.com/SeverinoCyber/30-days-of-code/tree/main/Hackerrank/day8>

Python3:

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 8: Dictionaries and Maps

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book.

Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.

After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq queries \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phoneNumber* in the format *name=phoneNumber*.

Sample Input

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
```

Change Theme

Language

Python 3

Exit Full Screen View

```
1 # Enter your code here. Read input from STDIN. Print output to STDOUT
2 n = int(input())
3 mapa = {}
4
5 for i in range(n):
6     name, phone = input().strip().split()
7     mapa[name] = phone
8
9
10 while True:
11     try:
12         buscar = input().strip()
13         if buscar in mapa:
14             print(f"{buscar}={mapa[buscar]}")
15         else:
16             print("Not found")
17     except EOFError:
18         break
```

Line: 18 Col: 14

Upload Code as File

Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 8: Dictionaries and Maps

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book.

Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.

After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq queries \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phoneNumber* in the format *name=phoneNumber*.

Sample Input

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
```

Exit Full Screen View

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Compiler Message

Success

Input (stdin)

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
harry
```

Expected Output

JavaScript(Node js):

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 8: Dictionaries and Maps

Exit Full Screen View

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book.
Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.
After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{queries} \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phone* Number in the format *name=phoneNumber*.

Sample Input

3
sam 99912222
tom 11122222
harry 12299933
sam
edward

Change Theme

Language

JavaScript (Node.js)

⌵

⌵

```
1 function processData(input) {
2   //Enter your code here
3   let lines = input.trim().split('\n');
4
5   let N = parseInt(lines[0].trim());
6   let mapa = {};
7
8   for (let i = 1; i <= N; i++) {
9     let [name, phone] = lines[i].trim().split(' ');
10    mapa[name] = phone;
11  }
12
13  for (let i = N + 1; i < lines.length; i++) {
14    let search = lines[i].trim();
15
16    if (search === "") continue;
17
18    if (mapa.hasOwnProperty(search)) {
19      console.log(search + "=" + mapa[search]);
20    } else {
21      console.log("Not found");
22    }
23  }
24
25  process.stdin.resume();
26  process.stdin.setEncoding("ascii");
27  _input = "";
28  process.stdin.on("data", function (input) {
29    _input += input;
30  });
31  process.stdin.on("end", function () {
32    processData(_input);
33  });
34}
```

Line: 24 Col: 10

Upload Codes File

Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

Prepare > Tutorials > 30 Days of Code > Day 8: Dictionaries and Maps

Exit Full Screen View

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book.
Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.
After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{queries} \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phone* Number in the format *name=phoneNumber*.

Sample Input

3
sam 99912222
tom 11122222
harry 12299933
sam
edward
harry

Upload Codes File

Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Facebook

Twitter

LinkedIn

Next Challenge

Test case 0

Compiler Message

Success

Input (stdin)

Download

3
sam 99912222
tom 11122222
harry 12299933
sam
edward
harry

Expected Output

Download

C#:

Problem

Submissions

Leaderboard

Discussions

Editorial

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book. Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.

After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{queries} \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phone.Number* in the format *name=phoneNumber*.

Sample Input

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
```

Change Theme Language C#

```
1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 class Solution {
5     static void Main(String[] args) {
6         /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution */
7         int N = Convert.ToInt32(Console.ReadLine());
8         Dictionary<string, string> mapa = new Dictionary<string, string>();
9
10        for(int i = 0; i < N; i++){
11            string[] input = Console.ReadLine().Split(' ');
12            string name = input[0];
13            string phone = input[1];
14
15            mapa[name] = phone;
16        }
17        string buscar;
18        while ((buscar = Console.ReadLine()) != null){
19            if (buscar == "") continue;
20
21            if (mapa.ContainsKey(buscar)){
22                Console.WriteLine(buscar+"="+mapa[buscar]);
23            }
24            else{
25                Console.WriteLine("Not found");
26            }
27        }
28    }
29 }
```

Line: 24 Col: 48

Upload Code as File Test against custom input

Run Code Submit Code

Problem

Submissions

Leaderboard

Discussions

Editorial

Objective

Today, we're learning about Key-Value pair mappings using a Map or Dictionary data structure. Check out the Tutorial tab for learning materials and an instructional video!

Task

Given n names and phone numbers, assemble a phone book that maps friends' names to their respective phone numbers. You will then be given an unknown number of names to query your phone book for. For each *name* queried, print the associated entry from your phone book on a new line in the form *name=phoneNumber*; if an entry for *name* is not found, print *Not found* instead.

Note: Your phone book should be a Dictionary/Map/HashMap data structure.

Input Format

The first line contains an integer, n , denoting the number of entries in the phone book. Each of the n subsequent lines describes an entry in the form of 2 space-separated values on a single line. The first value is a friend's name, and the second value is an 8-digit phone number.

After the n lines of phone book entries, there are an unknown number of lines of queries. Each line (query) contains a *name* to look up, and you must continue reading lines until there is no more input.

Note: Names consist of lowercase English alphabetic letters and are first names only.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{queries} \leq 10^5$

Output Format

On a new line for each query, print *Not found* if the name has no corresponding entry in the phone book; otherwise, print the full *name* and *phone.Number* in the format *name=phoneNumber*.

Sample Input

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
```

Line: 24 Col: 48

Upload Code as File Test against custom input

Run Code Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#) [Next Challenge](#)

Test case 0

Compiler Message

Success

Input (stdin)

```
3
sam 99912222
tom 11122222
harry 12299933
sam
edward
harry
```

Expected Output

```
Download
```